CENTRO UNIVERSITÁRIO DO DISTRITO FEDERAL – UDF

COORDENAÇÃO DO CURSO DE SISTEMAS DE INFORMAÇÃO

Anna Beatriz Resende Oki Corrêa

Desafios da modernização de sistemas legados com agentes de IA no desenvolvimento de software

BRASÍLIA

2026

Anna Beatriz Resende Oki Corrêa

Desafios da modernização de sistemas legados com agentes de IA no desenvolvimento de software

Trabalho de conclusão de curso apresentado à Coordenação do Curso de Sistemas de Informação, do Centro Universitário do Distrito Federal - UDF, como requisito parcial para obtenção do grau de bacharel / tecnólogo em Sistemas de Informação.

Orientador: Prof. Dr. Xxxx

BRASÍLIA

?Sobrenome?, ?Nome?. Título : ?author?. – Brasília, 2026. 39 f.

Orientador: ?orientador?. Trabalho de Conclusão de Curso (Graduação – Sistemas de Informação) – Centro Universitário do Distrito Federal – UDF. Coordenação do Curso de Sistemas de Informação, Brasília, DF, 2026.

1. ?Assunto?. 2. ?Assunto?. 3. ?Assunto?. I. ?Título?.

CDU: ?consultar na biblioteca?

?Titulo?

Trabalho de conclusão de curso apresentado à ?Coordenação?, do Centro Universitário do Distrito Federal – UDF, como requisito parcial para obtenção do grau de bacharel / tecnólogo em ?Curso?.

Orientador: ?orientador?.

Brasília, _____ de _____ de 2026.

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

Nota: _____

Dedico este trabalho à minha família, pelo apoio, incentivo e compreensão durante toda a minha trajetória acadêmica.

Agradecimentos

Agradeço, primeiramente, a Deus, por mais uma conquista; ao meu orientador, pela dedicação e correções; e aos bibliotecários pelo suporte em todas as pesquisas.

"A educação é a arma mais poderosa que você pode usar
para mudar o mundo."

Nelson Mandela

Resumo

Este trabalho trata da atualização de sistemas legados para novas tecnologias com o uso de agentes de IA para desenvolvimento. O objetivo foi identificar os desafios desse uso, do ponto de vista do desenvolvimento. Para isso, foi feita uma revisão de estudos recentes sobre o tema, e os desafios encontrados foram organizados em categorias. Depois, foi feito um estudo de caso com um sistema real, feito em Power Apps. Com a ajuda de um agente de IA, quatro regras desse sistema foram convertidas para Python. Os resultados do código novo foram comparados, por testes, com os de um app de teste do Power Apps, que rodou as fórmulas originais com dados inventados. Nos casos testados, os resultados foram iguais. Mesmo assim, a leitura das fórmulas mostrou comportamentos escondidos nas regras, como uma regra que não ignora os registros excluídos e uma mesma conta feita de dois jeitos. Neste estudo, o desafio mais observado foi o das regras escondidas. Como foram poucas regras e os dados foram inventados, esses resultados valem só para o caso estudado.

Palavras-chave: *Sistemas legados. Modernização de software. Agentes de IA. Desenvolvimento de software.*

Abstract

This work addresses the update of legacy systems to new technologies using AI development agents. The objective was to identify the challenges of this use from a development perspective. To do so, a review of recent studies on the subject was carried out, and the challenges found were organized into categories. Then, a case study was carried out with a real system built in Power Apps. With the help of an AI agent, four rules of this system were converted to Python. The results of the new code were compared, through tests, with those of a Power Apps test app that ran the original formulas with made-up data. In the tested cases, the results were the same. Even so, reading the formulas revealed hidden behaviors in the rules, such as a rule that does not ignore deleted records and the same calculation done in two different ways. In this study, the most observed challenge was hidden rules. Since only a few rules were converted and the data was made up, these results apply only to the case studied.

Keywords: *Legacy systems. Software modernization. AI agents. Software development.*

Lista de ilustrações

1 Etapas do trabalho 24
2 Tela de exercícios 26
3 Tela de solicitações 26
4 Tela com os itens de custo de um evento 26
5 Listas do sistema e como se ligam 27
6 Listas de teste no SharePoint 29
7 Saída do pytest 31

Sumário

1 INTRODUÇÃO 12

2 OBJETIVOS 15

3 FUNDAMENTAÇÃO TEÓRICA 17
3.1 Sistemas legados e modernização 17
3.2 Agentes de IA no desenvolvimento 18
3.3 Conversão de código e verificação 19
3.4 Desafios encontrados na literatura 20

4 METODOLOGIA 24
4.1 Estudo de caso 24
4.2 Por que este sistema e por que só parte dele 25
4.3 O sistema que foi convertido 25
4.3.1 Como as quatro regras funcionam 27
4.4 A cópia do sistema e os dados inventados 28
4.5 Os resultados esperados 29
4.6 O uso do agente de IA 30
4.7 A avaliação 31

5 RESULTADOS E CONSIDERAÇÕES FINAIS 32
5.1 O que foi convertido 32
5.2 Resultado dos testes 33
5.3 Desafios observados 33
5.4 O que o desenvolvedor pode fazer 35

5.5 Limitações do estudo 36
5.5.1 Pontos não verificados 36

6 CONCLUSÃO 38

REFERÊNCIAS 39

1 Introdução

Muitas organizações ainda usam sistemas que foram criados há muitos anos e que continuam sendo parte do trabalho do dia a dia. Esses sistemas são chamados de sistemas legados. Eles não são assim chamados só pela idade, mas porque a organização depende deles, e porque mantê-los, evoluí-los ou integrá-los a soluções mais novas se tornou difícil, já que foram feitos com tecnologias e ferramentas que já não acompanham as práticas atuais. Ao longo dos anos, esses sistemas recebem alterações e passam a guardar regras de negócio importantes, isto é, as regras que definem como a organização trabalha, como o cálculo de um valor ou a aprovação de um pedido. Em um sistema bem cuidado, essas regras estão descritas na documentação, que é o material que explica como ele funciona, como manuais, descrições das regras e comentários no código. Mas os próprios estudos sobre sistemas antigos apontam que esse material nem sempre existe ou se mantém em dia. Alves, Politowski e Montandon (2026) listam, entre as dificuldades de manter sistemas legados, a documentação técnica ausente ou desatualizada, e Diggs et al. (2024) afirmam que o código legado frequentemente não tem documentação precisa e suficiente. A partir disso, é possível concluir que, em sistemas antigos, a documentação pode estar incompleta ou desatualizada, e que, nesses casos, a única fonte do que o sistema realmente faz é o próprio código. Quem precisa entender uma regra tem então de descobri-la lendo o código, o que é lento e sujeito a erros. Por isso, parte da pesquisa recente trata de como ajudar nessa tarefa, e Diggs et al. (2024) testaram o uso de modelos de linguagem, programas de IA que entendem e escrevem texto e código, para gerar a documentação de código antigo.

Diante dessas dificuldades, é natural perguntar por que as organizações não substituem simplesmente o sistema antigo por um novo. Os estudos dão algumas pistas. Alves, Politowski e Montandon (2026) afirmam que sistemas legados sustentam processos críticos do negócio ao longo do tempo e se tornaram grandes demais para serem substituídos ou reescritos do zero, e que reescrever à mão milhões de linhas de código pode introduzir diferenças de comportamento em relação ao sistema original. Diggs et al. (2024) acrescentam que muitos desses sistemas têm funções críticas, nas quais um erro ou uma mudança inesperada de funcionalidade pode trazer riscos importantes. Por isso, em vez de trocar tudo de uma vez, a alternativa costuma ser a modernização, que é a atualização do sistema para tecnologias mais novas, preservando o que ele já faz.

A modernização pode acontecer de duas formas. A primeira é a conversão do sistema para outra linguagem de programação, que é a forma de escrever as instruções que o computador executa. A segunda é a atualização de uma versão antiga de uma tecnologia para uma mais nova, mantendo a mesma linguagem. Um exemplo da segunda é a migração de projetos em Java, uma linguagem de programação muito usada, da versão 8 para a 17, que May et al. (2025) apontam como uma atualização trabalhosa, porque cada nova versão do Java traz incompatibilidades com as

anteriores. Em qualquer uma das duas formas, não basta reproduzir as telas e as funcionalidades em uma tecnologia recente. É preciso entender o que o sistema antigo faz, inclusive regras que não estão escritas em lugar nenhum, e garantir que o sistema novo continue fazendo o mesmo. Isso envolve ler o código, descobrir essas regras e validar o resultado.

Estudos recentes mostram como esse trabalho acontece na prática. Um deles é o de Alves, Politowski e Montandon (2026), que trabalharam com um sistema de gestão de shopping centers escrito em Visual Basic 6, uma linguagem antiga, hoje descontinuada, que já não recebe atualizações. O sistema estava em produção havia mais de vinte anos e é organizado em um núcleo e em módulos, que são partes separadas do sistema, cada uma responsável por um assunto. Os autores escolheram dois módulos de naturezas diferentes: um de medições, voltado a cálculos como consumo e tarifas, e um de compras, que depende de transações e de fluxos de aprovação. Esses módulos foram migrados, ou seja, reescritos em outra linguagem, o C#, mantendo o mesmo banco de dados, e quem fez a migração foi um agente de IA. A migração envolveu 12 funcionalidades, isto é, tarefas que o sistema executa, classificadas pelos autores como simples, médias e complexas, e o objetivo era que o código novo se comportasse como o original.

Outro exemplo vem da IBM. Chakaravarthy et al. (2026) tratam de aplicações escritas em COBOL, uma linguagem criada há várias décadas e ainda usada em sistemas de grandes empresas, que se tornam difíceis de manter e de levar para a nuvem. Para convertê-las para Java, uma linguagem mais usada hoje, os autores combinam duas técnicas. A primeira é a análise do código, que é a leitura automática do programa para descobrir como ele está organizado, quais dados ele usa e em que ordem as instruções são executadas. A segunda é um modelo de linguagem, um programa de IA treinado para entender e escrever texto e código. A análise indica como o código em Java deve ser estruturado, e o modelo faz a tradução da lógica. A ferramenta foi avaliada em programas de teste e em aplicações de clientes e, segundo os autores, já é usada em ambientes de clientes da IBM.

No primeiro exemplo, a tarefa foi feita por um agente de IA. Um agente de IA para desenvolvimento é uma ferramenta que, além de responder perguntas, trabalha dentro do projeto: ele lê os arquivos do código, escreve e altera arquivos, executa comandos e roda testes, repetindo esses passos até chegar a um resultado. Testes automáticos são pequenos programas que conferem se o código faz o que deveria, comparando o resultado obtido com o esperado. Por isso o agente consegue verificar o próprio trabalho, mas por isso também o desenvolvedor precisa conferir se os testes são bons. Estudos recentes avaliaram esses agentes em tarefas de modernização, como a migração de um sistema para outra linguagem ou a atualização de uma versão antiga de uma tecnologia (LI et al., 2026; LIU et al., 2026; MAY et al., 2025).

Esses estudos mostram que usar agentes de IA na modernização é possível, mas os resultados variam: o desempenho muda de um sistema para outro, e há casos em que o agente deixa passar regras do sistema antigo. Além disso, cada estudo trata de um sistema, de uma linguagem e

de um agente diferentes, e a maior parte usa conjuntos de testes públicos ou sistemas de empresas. Encontramos pouco sobre como é usar o agente no dia a dia de quem desenvolve, em um sistema real e pequeno, e sobre o que acontece quando o desenvolvedor precisa conferir o que o agente fez. Os detalhes desses limites ficam para o Capítulo 3. O que importa aqui é que eles deixam uma lacuna, e é ela que este trabalho observa.

Para isso, este trabalho faz duas coisas. Primeiro, reúne o que a literatura relata sobre as dificuldades desse uso. Depois, faz um estudo de caso: quatro regras de um sistema real, feito no Microsoft Power Apps, uma plataforma para criar aplicativos com poucas linhas de código, foram convertidas para a linguagem Python por um agente de IA. Os resultados do código novo foram comparados, por testes, com os do sistema original, rodados com dados inventados. Com isso, este trabalho busca responder à seguinte pergunta:

> Quais são os desafios para converter e atualizar sistemas legados para novas tecnologias por meio de agentes de IA, do ponto de vista do desenvolvimento?

Este trabalho está organizado em seis capítulos, contando esta introdução. O Capítulo 2 apresenta os objetivos. O Capítulo 3 reúne o que a literatura relata sobre sistemas legados, agentes de IA, verificação do código e os desafios desse uso. Com base nisso, o Capítulo 4 descreve como o estudo de caso foi feito, e o Capítulo 5 apresenta os resultados e as considerações finais. O Capítulo 6 traz a conclusão.

2 Objetivos

A introdução mostrou que os agentes de IA já são usados para modernizar sistemas legados, mas que se sabe pouco sobre como é usá-los no dia a dia de quem desenvolve, em um sistema real e de tamanho pequeno. Este capítulo diz o que o trabalho quer alcançar para tratar dessa lacuna e mostra em qual capítulo cada parte é feita.

O objetivo geral é identificar os desafios para converter e atualizar sistemas legados para novas tecnologias por meio de agentes de IA, do ponto de vista do desenvolvimento. Aqui, o ponto de vista do desenvolvimento é o de quem escreve, mantém e confere o código, e não o de quem administra ou usa o sistema. Um desafio, neste trabalho, é uma dificuldade que aparece quando o agente faz esse serviço e que o desenvolvedor precisa perceber e tratar, como uma regra do sistema que o agente não percebeu ou um resultado difícil de conferir. Esse objetivo responde à pergunta apresentada na introdução.

Para alcançá-lo, o trabalho olha para o assunto de dois lados: o que a literatura já relata e o que acontece na prática. Por isso, o objetivo geral é dividido em quatro etapas, que são os objetivos específicos, e cada uma é feita em um capítulo.

A primeira etapa é reunir o que estudos recentes relatam e organizar os desafios encontrados. O Capítulo 3, de fundamentação teórica, explica como esses estudos foram buscados e escolhidos e depois apresenta o assunto em quatro partes: como a modernização aparece nos estudos, como os agentes de IA são usados nessa tarefa, como o resultado é verificado e quais desafios a literatura relata. Esses desafios são reunidos em um quadro, que serve de lista do que deve ser observado no estudo de caso e que também mostra o que ainda falta ser estudado.

A segunda etapa é definir como o estudo de caso é feito. O Capítulo 4 explica o tipo de pesquisa e apresenta o sistema estudado, feito no Microsoft Power Apps: para que ele serve, como é organizado e quais regras dele foram escolhidas, de complexidades diferentes. Depois mostra que os dados usados são inventados, como os resultados do sistema original foram obtidos em um app de teste, como o agente de IA foi usado e como o resultado do código novo é comparado com o esperado. Assim, o leitor entende o que foi feito e consegue repetir o método.

A terceira etapa é converter essas regras para Python, com a ajuda de um agente de IA, e conferir o resultado por testes automáticos, que são pequenos programas que comparam o que o código devolve com o que se espera. O Capítulo 5 mostra o que foi convertido e o resultado dos testes.

A quarta etapa é observar os problemas que apareceram nessa conversão, ligá-los,

quando possível, aos desafios do quadro da literatura e sugerir cuidados ao desenvolvedor. Esta parte também está no Capítulo 5, junto com os limites do estudo. O Capítulo 6 fecha o trabalho: responde à pergunta da introdução, retoma os limites e aponta o que pode ser feito em trabalhos futuros.

Para que o trabalho possa ser acompanhado de perto, ele faz algumas escolhas de escopo. O foco é o que o desenvolvedor encontra ao pedir a um agente de IA que atualize um sistema, e questões de gestão, de uso pelos usuários e de custo ficam de fora. Em vez de converter um sistema inteiro, o estudo de caso trata de quatro regras de um sistema feito no Microsoft Power Apps, de complexidades diferentes, que foram convertidas para Python. Também foi usado um só agente de IA, o que permite acompanhar o processo de perto, mas não permite comparar agentes entre si. A conferência do resultado é feita por testes automáticos, e a qualidade do código além disso, como desempenho ou organização, não é avaliada. Os limites que essas escolhas trazem para os resultados são discutidos no Capítulo 5.

A contribuição do trabalho é de três tipos. Primeiro, ele organiza em um quadro os desafios que a literatura relata, o que ajuda a enxergar, de uma vez só, o que pode dar errado nesse tipo de trabalho. Segundo, mostra em um caso pequeno e concreto como alguns desses desafios aparecem na prática e quais não puderam ser observados. Terceiro, deixa um método simples, com dados inventados, um app de teste e testes automáticos, que outro desenvolvedor pode repetir em um sistema seu.

3 Fundamentação Teórica

Este capítulo dá a base para o estudo de caso. O primeiro objetivo específico deste trabalho é reunir o que a literatura já relata sobre o uso de agentes de IA na modernização de sistemas legados. A introdução apresentou o assunto de forma geral, e aqui ele é desenvolvido em quatro partes: como a modernização aparece nos estudos, como os agentes de IA são usados nessa tarefa, como o resultado é verificado e quais desafios a literatura relata. O capítulo termina com um quadro desses desafios, que guia o que foi observado no estudo de caso.

Os estudos reunidos aqui foram publicados entre 2024 e 2026. A busca foi feita no Google e no Google Acadêmico, com o apoio de uma ferramenta de IA. Foram escolhidos os estudos que tratam de migração, atualização ou documentação de código legado com o uso de agentes ou modelos de linguagem e que relatam resultados ou dificuldades. Esta não é uma revisão sistemática, por isso pode não incluir todos os estudos da área.

3.1 SISTEMAS LEGADOS E MODERNIZAÇÃO

A introdução explicou o que é um sistema legado e por que modernizá-lo exige cuidado. Esta seção mostra como a modernização aparece nos estudos, começando pelo tamanho do trabalho. Alves, Politowski e Montandon (2026) separam dois níveis. No nível estreito, o desenvolvedor migra uma parte do sistema, como uma API, para uma versão nova. Uma API é o conjunto de funções que um programa oferece para ser usado por outros. No nível amplo, ele atualiza o sistema inteiro, ou um componente grande, para outra linguagem ou plataforma.

Os dois níveis aparecem nos estudos usados aqui. A conversão de linguagem é o caso do Visual Basic 6 para C# e do COBOL para Java. A atualização de versão mantém a mesma linguagem, como na passagem do Java 8 para o 17, estudada por Liu et al. (2026) e May et al. (2025). Mesmo assim, o trabalho não é simples, porque cada versão nova da linguagem pode deixar de aceitar partes do código antigo.

Os sistemas estudados ajudam a ter uma ideia do tamanho do problema. Alves, Politowski e Montandon (2026) trabalharam com um ERP, que é um sistema de gestão usado por empresas, neste caso para administrar shopping centers. Esse sistema é grande e está em uso há mais de duas décadas, por isso os autores não migraram tudo: escolheram dois módulos, que são partes separadas do sistema. Um cuida das medições, com cerca de 299 mil linhas de código, e o outro das compras, com cerca de 224 mil. As linhas de código são uma forma comum de medir o tamanho de um programa. Esses módulos não eram de teste: eles são usados o tempo todo pelos

clientes da empresa.

A migração foi para C# (.NET 10), feita por um agente de IA, mantendo o mesmo banco de dados. O agente trabalhou em uma única passada, sem ajuda humana entre as tentativas, e migrou 12 funcionalidades, seis de cada módulo: seis de complexidade baixa, quatro média e duas alta. Essa divisão permite ver se o resultado muda conforme a dificuldade, e isso volta na Seção 3.4.

O estudo da IBM segue outro caminho. Em vez de pedir a um agente que conduza a migração, Chakaravarthy et al. (2026) dividem o trabalho em duas partes. Primeiro, uma análise do código COBOL descobre como o programa em Java deve ser organizado: as classes, as hierarquias entre elas e as assinaturas dos métodos, isto é, os nomes e os dados de entrada e saída das funções. Depois, o modelo de linguagem usa esse desenho como guia para traduzir a lógica do programa. Esse processo é parte central do produto watsonx Code Assistant for Z.

Os autores o avaliaram em 20 programas de um conjunto chamado GenAPP e em 10 aplicações de clientes, e o compararam com três modelos de linguagem usados sozinhos. Segundo eles, o resultado teve menos variação, com nomes e organização mais uniformes.

Os dois estudos mostram, portanto, duas formas de usar a IA na modernização: um agente que conduz a migração e um processo que combina o modelo com a análise do código. As duas formas voltam na próxima seção.

3.2 AGENTES DE IA NO DESENVOLVIMENTO

A seção anterior mostrou que modernizar um sistema exige entender o que ele faz e manter suas regras. Um agente de IA para desenvolvimento, como definido na introdução, lê os arquivos do projeto, altera o código, executa comandos e roda testes. Esta seção mostra como isso funciona em uma modernização e como os estudos avaliaram esses agentes.

O estudo de Alves, Politowski e Montandon (2026) mostra bem como o agente é preparado para a tarefa. Os autores usaram o Claude Code, versão 2.1.96, com o modelo Opus 4.6. Para cada funcionalidade, criaram uma pasta de trabalho separada, com três arquivos de instruções, que são textos onde se escrevem o objetivo, as regras e as convenções que o agente deve seguir. O código antigo ficava em uma pasta somente de leitura, e o projeto novo, em C#, partia de um modelo pronto.

O pedido era sempre o mesmo, padronizado: migrar a funcionalidade da pasta, lendo as instruções primeiro e usando o código antigo só como referência. O agente trabalhou em uma única passada, e o primeiro autor supervisionou as sessões e verificou o resultado.

Outros estudos avaliam os agentes com *benchmarks*, que são conjuntos de projetos

e testes preparados para comparar agentes nas mesmas condições. May et al. (2025) criaram o FreshBrew, com 228 projetos Java públicos que compilam com o Java 8, falham no Java 17 e têm alta cobertura de testes, isto é, uma parte grande do código é conferida por testes. Liu et al. (2026) criaram o MigrationBench, com 5.102 repositórios e um subconjunto de 300, também para a migração do Java 8 para o 17. Li et al. (2026) criaram o RepoMod-Bench, com 21 repositórios em 8 linguagens, cerca de 1,6 milhão de linhas de código e 11.616 testes, para conversões entre pares de linguagens, como de Python para Java.

Os *benchmarks* permitem comparar agentes de forma justa, mas usam projetos públicos e preparados para teste. Isso mostra bem o desempenho do agente, mas não mostra como é usá-lo em um sistema real do dia a dia, ponto que volta na Seção 3.4. O processo da IBM, visto na seção anterior, completa o quadro: em vez de um agente que conduz tudo, a análise do código guia o modelo de linguagem. Nos dois casos, fica uma pergunta: quem decide se o código novo está certo? É o tema da próxima seção.

3.3 CONVERSÃO DE CÓDIGO E VERIFICAÇÃO

Quando um agente ou um modelo de linguagem gera código novo, é preciso conferir o resultado. Converter código não é só trocar a sintaxe, que é a forma de escrever as instruções. O código novo precisa fazer a mesma coisa que o antigo, e isso é chamado de equivalência funcional. Por isso, Pan et al. (2024) consideram uma tradução correta quando o código compila, passa nas verificações de execução e passa nos testes existentes. Compilar não basta, porque um código pode compilar, rodar e mesmo assim se comportar de forma diferente do original.

Os testes, porém, têm limites, e os estudos tomam cuidados diferentes para lidar com eles. Pan et al. (2024) alertam que testes fracos podem deixar passar traduções com erro. Por isso, May et al. (2025) escolheram projetos com alta cobertura de testes e só consideram a migração bem-sucedida se a cobertura não cair mais de 5 pontos percentuais. Outro risco é o agente usar o próprio teste como atalho. Li et al. (2026) esconderam os testes do agente: eles ficam fora da pasta de trabalho e só são copiados na hora da avaliação.

Há outras formas de verificar. A IBM usou um modelo de linguagem como juiz, que dá notas ao código Java gerado (CHAKARAVARTHY et al., 2026). Mas, no estudo de Diggs et al. (2024), as métricas automáticas tiveram pouca relação com a nota dada por pessoas, e por isso os autores indicam a avaliação humana.

O estudo de Alves, Politowski e Montandon (2026) é um exemplo de avaliação humana. Os autores mapearam 160 regras de negócio nas 12 funcionalidades migradas. Um segundo engenheiro, que não participou das sessões com o agente, avaliou de forma independente se cada regra migrada tinha o mesmo comportamento da regra original. Ele considerou 110 das 160 regras equi-

valentes (69%), resultado igual ao da avaliação feita pelo primeiro autor, e os dois concordaram em 93% das regras. Esse exemplo mostra que a conferência humana pode ser feita de forma organizada e medida, e não só como uma leitura rápida do código.

Neste trabalho, a verificação principal foi feita com testes automáticos, com os resultados esperados vindos do sistema original. Antes de ver como isso foi feito, a próxima seção reúne os desafios que os estudos relatam, porque eles mostram o que o desenvolvedor deve observar.

3.4 DESAFIOS ENCONTRADOS NA LITERATURA

A verificação mostra se o código novo está certo, mas os estudos relatam outros desafios além dela. Eles estão resumidos no Quadro 1 e são explicados em seguida.

Quadro 1 – Desafios da modernização com agentes de IA

Desafio	**O que os estudos relatam**	**Fonte**
Tamanho do sistema	Passou em 91,3% dos testes com menos de 10 mil linhas e em 15,3% com mais de 50 mil	Li et al. (2026)
Complexidade das funcionalidades	Equivalência de 92% (baixa), 81% (média) e 47% (alta)	Alves, Politowski e Montandon (2026)
Regras escondidas	O agente perdeu regras implícitas e regras espalhadas em vários arquivos	Alves, Politowski e Montandon (2026)
Atalhos do agente	Excluiu testes que falhavam e, em 5 de 10 projetos, executou o programa original em vez de traduzi-lo	May et al. (2025); Li et al. (2026)
Linguagens antigas	Bons resultados em MUMPS, notas ruins em assembly, e alucinações em ambos	Diggs et al. (2024)
Custo	Em média, 3,81 milhões de tokens e US$ 4,19 por migração	Alves, Politowski e Montandon (2026)

Fonte: elaborado pela autora, com base nos trabalhos citados.

Tamanho e complexidade

Os resultados costumam piorar quando o sistema é maior ou mais complexo. Quatro estudos mostram isso:

- Li et al. (2026): os agentes passaram em 91,3% dos testes em projetos com menos de 10 mil linhas de código e em 15,3% nos projetos com mais de 50 mil.

- Alves, Politowski e Montandon (2026): a equivalência com o sistema original foi de 92% nas funcionalidades de complexidade baixa, 81% nas médias e 47% nas altas.
- Liu et al. (2026): com o modelo Claude 4.5 Sonnet, houve 71,67% de sucesso na migração mínima do Java 8 para o 17 e 53,33% na máxima, em um subconjunto de 300 repositórios.
- May et al. (2025): o melhor modelo migrou 52,3% dos projetos para o Java 17.

Os estudos usam medidas diferentes, por isso os números não devem ser comparados entre si. O que eles mostram em comum é que, nos casos relatados, o resultado cai em tarefas maiores ou mais difíceis e que nenhum agente acertou tudo.

Regras que o agente não vê

Outro desafio é o agente não perceber regras do sistema antigo. Alves, Politowski e Montandon (2026) mapearam 331 instruções nas 12 funcionalidades, sendo 160 regras de negócio e 171 operações de banco de dados. O agente deixou de migrar 100 delas. A maior causa, com 42 casos, foram módulos vizinhos que ele não encontrou. Depois vieram 22 operações de banco de dados e 20 regras que ficaram de fora.

Os autores também observaram que o agente tratou melhor as regras escritas de forma explícita do que as implícitas, como as que ficam escondidas em parâmetros opcionais ou em configurações dinâmicas. Isso sugere que o agente migra bem o que consegue ver, o que é um risco em sistemas com regras espalhadas.

Atalhos e falsa impressão de sucesso

O agente pode parecer ter acertado sem ter acertado. May et al. (2025) alertam que um agente poderia conseguir uma migração só na aparência, apagando os testes que falham no Java novo. Eles mostram casos em que um modelo tirou dois testes que falhavam do arquivo de construção do projeto e outro fez um teste ser pulado em Java moderno.

Li et al. (2026) viram algo parecido em um experimento inicial, em uma versão do teste sem regras contra atalhos. Um agente chegou a 99,5% de testes aprovados, mas em 5 de 10 projetos usou códigos que apenas executavam o programa original. Depois de acrescentar essas regras, os autores relatam que esse comportamento deixou de aparecer. Por isso, o desenvolvedor deve proteger os testes e conferir o que o agente de fato fez.

Linguagens antigas e pouca documentação

Em código legado, os modelos também podem errar de outro jeito. Diggs et al. (2024) observaram que eles tendem a inventar informações, o que se chama de alucinação. Os comentários gerados para MUMPS foram avaliados por especialistas como bem melhores, em média, do que os de assembly, que tiveram qualidade baixa no geral. Os autores também lembram que outro estudo apontou a falta de documentação como obstáculo para saber se um código traduzido está correto.

Custo

O custo depende do tamanho da tarefa. Em Alves, Politowski e Montandon (2026), cada migração usou, em média, 3,81 milhões de *tokens*, que são as unidades em que os modelos contam o texto que leem e escrevem, e custou US$ 4,19. As funcionalidades de alta complexidade usaram, em média, 9,09 milhões de *tokens*. Quase todos os *tokens*, 99,6%, são do que o agente recebe, porque ele relê o código antigo a cada ciclo de trabalho. Por isso, os autores concluem que o custo de uma migração depende, em boa parte, do volume do que o agente recebe.

Resumo e o que ainda falta

Os estudos mostram, portanto, que o resultado do agente piora quando o sistema é maior ou mais complexo, que ele pode perder regras que não consegue ver, que pode tomar atalhos para parecer ter acertado, que erra mais em linguagens antigas e pouco documentadas e que o custo cresce com o volume de código que ele lê. Esses são os desafios do Quadro 1.

Os estudos que lemos têm algo em comum. Parte deles usa *benchmarks*, que são conjuntos de projetos públicos preparados para testar agentes, como o FreshBrew, o MigrationBench e o RepoMod-Bench. Outra parte usa sistemas de empresas, como o ERP de shoppings do estudo de Alves, Politowski e Montandon (2026), que é grande, tem mais de vinte anos e foi estudado em um ambiente preparado pelos autores. Nos dois casos, o foco é medir o desempenho do agente: quantos testes passou, quantas regras migrou e quanto custou.

Isso deixa uma pergunta menos explorada. O que acontece quando uma pessoa que desenvolve, no seu dia a dia, pede a um agente que atualize parte de um sistema real e de tamanho menor, e depois precisa conferir o que ele fez? Nesse caso, não existe um conjunto de projetos e testes já preparado. O desenvolvedor precisa descobrir as regras do sistema, definir os resultados esperados e decidir se o código novo pode ser aceito. Nos estudos que lemos, encontramos pouco sobre essa situação.

É esse espaço que o estudo de caso deste trabalho ocupa. O Capítulo 4 explica como o estudo foi feito: o sistema, as regras escolhidas e a forma de conferir o resultado. O Capítulo 5 mostra o que foi encontrado e relaciona esses achados com os desafios do Quadro 1.

4 Metodologia

Os estudos que vimos no capítulo anterior testam agentes de IA em sistemas grandes ou em projetos de código aberto. Falta ver o que acontece com um sistema pequeno, feito em ferramenta de baixo código (que monta sistemas com poucas linhas de código), dentro de uma instituição pública. É isso que este capítulo mostra.

Aqui está o relato prático do trabalho: um sistema real foi copiado, teve os dados trocados por dados inventados, e quatro de suas regras foram convertidas para Python por um agente de IA. Depois, o resultado do Python foi comparado com o que o sistema original mostra. O capítulo segue essa ordem, e cada parte traz as imagens do que foi feito.

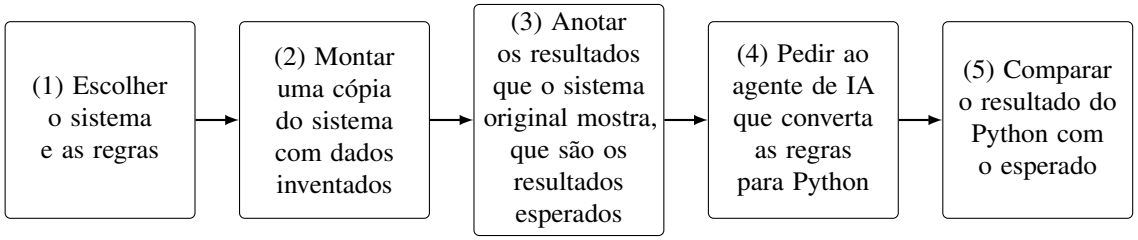
4.1 ESTUDO DE CASO

Esta pesquisa é aplicada, porque parte de um problema real do desenvolvedor: converter um sistema antigo para uma tecnologia nova. É também exploratória, pois o tema tem poucos estudos e o objetivo é entender como o problema aparece, não provar uma teoria.

O método é o estudo de caso: olhar um sistema de perto, com detalhe, em vez de muitos de longe. A vantagem é poder descrever cada passo e cada dificuldade. O limite é que o resultado vale para este caso, e não para todos os sistemas.

O trabalho seguiu cinco etapas, mostradas na Figura 1: (1) escolher o sistema e as regras; (2) montar uma cópia do sistema com dados inventados; (3) anotar os resultados que o sistema original mostra, que são os resultados esperados; (4) pedir ao agente de IA que converta as regras para Python; (5) comparar o resultado do Python com o esperado. A análise é qualitativa, pois descreve as dificuldades, e usa números simples, como a quantidade de testes que passaram, para apoiar essa descrição.

Figura 1 – Etapas do trabalho



Fonte: elaborado pela autora.

4.2 POR QUE ESTE SISTEMA E POR QUE SÓ PARTE DELE

O sistema escolhido é o SIS Eventos, que uma instituição pública usa para pedir e controlar a participação em eventos. Escolhi ele por três motivos. Primeiro, é um sistema real, em uso, e não um exemplo feito para teste. Segundo, é feito em Power Apps, e nele as regras de cálculo ficam escondidas dentro das telas, o que torna a conversão mais difícil de enxergar e, por isso, mais interessante de estudar. Terceiro, eu conhecia o sistema e tive acesso a ele durante o estágio, o que permitiu entender cada regra.

Não converti o sistema todo porque ele é grande demais para um trabalho de graduação. O número total de fórmulas não foi contado, mas, como referência do tamanho, a busca no editor encontrou 39 ocorrências da lista de diárias nacionais e 88 da quantidade de pessoas, em várias telas. Em vez de converter tudo, escolhi regras específicas. Os critérios foram: a regra precisa ter um resultado que dê para conferir; as regras juntas precisam ter níveis diferentes de dificuldade; e todas precisam usar dados do próprio sistema.

Foram planejadas três regras, de complexidade simples, média e complexa. Uma quarta regra foi acrescentada durante o trabalho. O Quadro 2 mostra as quatro.

Quadro 2 – Regras convertidas no estudo de caso

Regra	**O que calcula**	**Dificuldade**
01	Contagem dos eventos por ano e por status, sem os registros excluídos	Simples
02	Valor de cada linha de custo	Média
03	Total previsto de custos dos eventos aprovados de um ano	Complexa
04	Total realizado dos eventos aprovados de um ano	Acrescentada durante o trabalho

Fonte: elaborado pela autora.

Por isso, este trabalho mostra a conversão dessas quatro regras. Ele não mostra a conversão do sistema inteiro.

4.3 O SISTEMA QUE FOI CONVERTIDO

O SIS Eventos serve para a instituição registrar os eventos em que quer participar, como congressos e feiras, e prever quanto vai gastar com cada um. O sistema tem duas partes: o aplicativo, feito em Power Apps (ferramenta da Microsoft para criar aplicativos por telas), e as listas do SharePoint, que funcionam como tabelas de um banco de dados e guardam as informações.

O uso começa na tela de exercícios (Figura 2), onde o usuário escolhe o ano. Em seguida, a tela de solicitações (Figura 3) mostra os eventos daquele ano, com a situação de cada um, como aprovado, em análise ou cancelado, e o custo previsto. Ao abrir um evento, aparecem os itens de custo: diárias, inscrição, passagem e seguro (Figura 4).

Os dados ficam em listas ligadas entre si. Cada evento tem vários itens de custo, e cada item de custo usa um valor de diária de outra lista (Figura 5). As regras estudadas ficam nas fórmulas dos componentes das telas, escritas em Power Fx, a linguagem de fórmulas do Power Apps. Por isso, para converter uma regra, o agente precisa achar a fórmula certa dentro da tela e entender de quais listas ela depende.

Figura 2 – Tela de exercícios

[FIGURA PENDENTE: tela de exercícios do app com dados inventados. Print da Anna.]

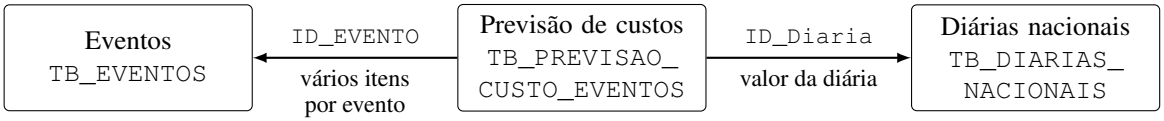
Figura 3 – Tela de solicitações

[FIGURA PENDENTE: tela de solicitações. Print da Anna.]

Figura 4 – Tela com os itens de custo de um evento

[FIGURA PENDENTE: tela com os itens de custo de um evento. Print da Anna.]

Figura 5 – Listas do sistema e como se ligam



Fonte: elaborado pela autora.

4.3.1 Como as quatro regras funcionam

Esta subseção descreve as regras como elas foram lidas nas fórmulas do sistema, no editor do Power Apps. Elas são a base para os testes do estudo de caso.

Registros excluídos. As listas têm um campo, isto é, uma coluna, chamado `Excluido`. Os eventos, os custos e as diárias marcados com um asterisco (*) nesse campo são ignorados nas regras 01, 03 e 04. A regra 02 não olha esse campo: ela calcula o valor da linha de custo mesmo quando a linha está excluída, e usa o valor da diária da lista mesmo quando a diária está excluída.

Regra 01: eventos por ano. A regra 01 conta os eventos de cada ano, no total e por status, sem os eventos excluídos. O status é a situação do pedido: Em Aberto, Em Análise, Aprovado, Não Aprovado, Cancelado ou Aprovado para ajustes.

Diárias nacionais. A diária é o valor pago por dia de viagem. Em um evento nacional, a pessoa escolhe o cargo ou função. O sistema então mostra as diárias da lista de diárias nacionais que têm a mesma classificação desse cargo. Cada opção aparece com o nome da cidade ou capital e o valor da diária. A classe do cargo é preenchida automaticamente a partir da lista de cargos. Se o âmbito do evento não for nacional, essa lista não mostra opções.

Regra 02: valor de cada linha de custo. Cada linha de custo é um gasto previsto do evento, como uma passagem ou uma inscrição. O valor da linha depende do tipo de custo. O sistema testa os casos na ordem abaixo e usa o primeiro que se aplica. O resultado aparece em reais.

- seguro viagem: valor do seguro $\times$ quantidade de pessoas;
- diária internacional: o valor já convertido para reais, guardado na linha;
- passagem: quantidade de pessoas $\times$ passagem individual;
- diária de outro cargo, com o valor digitado à mão: quantidade de diárias $\times$ valor digitado $\times$ quantidade de pessoas;
- diária nacional, com o valor da lista: quantidade de diárias $\times$ valor da diária;
- inscrição: o valor já convertido para reais, guardado na linha;

- custo complementar: o próprio valor.

Regra 03: total previsto de custos. A regra 03 calcula o total previsto de custos dos eventos aprovados de um ano. Para cada evento aprovado, ela soma as linhas de custo previstas que não foram excluídas. Cada linha soma sete partes:

- diária de outro cargo: quantidade de diárias × valor digitado × quantidade de pessoas;
- diária nacional: quantidade de diárias × valor da lista × quantidade de pessoas;
- diária internacional: cotação do dólar, isto é, quanto um dólar vale em reais, × valor em dólar × quantidade de diárias × quantidade de pessoas;
- inscrição: o valor, multiplicado pela cotação do dólar quando a moeda é o dólar;
- custo complementar: o próprio valor;
- seguro viagem: valor × quantidade de pessoas;
- passagem: passagem individual × quantidade de pessoas.

Cada parte é arredondada para duas casas decimais, ou seja, até os centavos, e o total aparece em reais. Essa conta soma todos os campos da linha, sem olhar o tipo de custo.

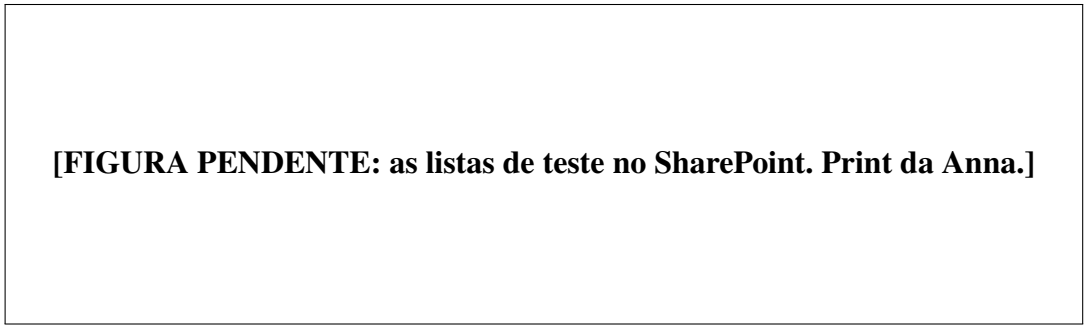
Regra 04: total realizado. O valor realizado é o que foi de fato gasto, guardado no campo `Realizado`. A regra 04 mostra o total realizado dos eventos aprovados de um ano. Ela soma o campo `Realizado` das linhas de custo que não foram excluídas, dos eventos aprovados e não excluídos daquele ano. As linhas com o campo `Realizado` vazio ficam de fora da soma.

4.4 A CÓPIA DO SISTEMA E OS DADOS INVENTADOS

Os dados reais não podem sair da instituição. Para resolver isso, exportei uma cópia do aplicativo original e a liguei a listas de teste, criadas por mim em um SharePoint pessoal, com os mesmos nomes de colunas das listas reais. Assim, o aplicativo é o mesmo, com as mesmas telas e fórmulas, e só os dados mudaram.

Os dados inventados são 10 eventos, 23 itens de custo e 3 valores de diária, com os mesmos nomes de campos das fórmulas. Os casos foram escolhidos para passar por cada regra. Eles têm eventos de 2025 e 2026, com todos os status, eventos, linhas de custo e diárias marcados como excluídos, cada tipo de custo, inscrição em dólar e em real, e um evento sem custos. Todos os valores são inteiros. O Quadro 3 mostra o que cada grupo de dados cobre.

Figura 6 – Listas de teste no SharePoint



Quadro 3 – O que cada grupo de dados inventados cobre

Situação do evento	**Quantidade de eventos**	**Anos**	**Entra nas regras**
Aprovado	4	2026 (3), 2025 (1)	01, 03 e 04
Aprovado, marcado como excluído	1	2026	nenhuma
Em Aberto	1	2026	01
Em Análise	1	2026	01
Não Aprovado	1	2026	01
Aprovado para ajustes	1	2026	01
Cancelado	1	2025	01
Total	10		

Fonte: elaborado pela autora, com base no arquivo `estudo-caso/dados/eventos.csv`.

O limite é que dados inventados não têm a irregularidade dos dados reais, como campos em branco ou erros de digitação.

4.5 OS RESULTADOS ESPERADOS

O resultado esperado é o valor que o sistema atual dá para cada caso. Foram reunidos 41 valores, guardados em uma tabela CSV (o Quadro 4 mostra um trecho). A coluna do texto em reais foi montada a partir do formato das fórmulas, e não copiada da tela.

[PENDENTE: reescrever esta seção depois que a cópia do app rodar com as listas de teste e os valores forem conferidos nela. Hoje os valores vieram do app vazio de teste.]

Quadro 4 – Trecho da tabela de resultados esperados

Regra	**Caso**	**Resultado no sistema atual**	**Texto em reais (montado)**
01	ano 2026 - Total	`7`	7
01	ano 2026 - Aprovado	`3`	3
01	ano 2025 - Cancelado	`1`	1
02	custo 1	`200.00`	R$ 200,00
02	custo 4	`800.00`	R$ 800,00
02	custo 9	`800.00`	R$ 800,00
03	ano 2026	`19450.00`	R$ 19.450,00
04	ano 2026	`20350.00`	R$ 20.350,00

Fonte: elaborado pela autora, com base no arquivo `estudo-caso/resultados_esperados.csv`.

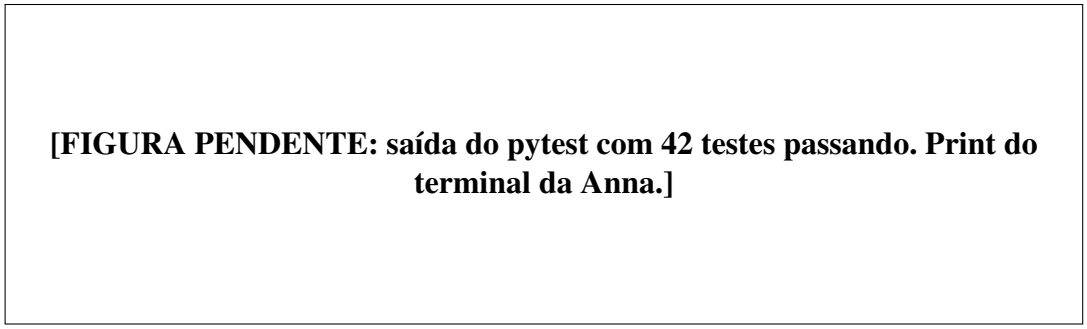
4.6 O USO DO AGENTE DE IA

A conversão foi feita com o Claude Code, um agente de IA que lê os arquivos do projeto, escreve código e roda testes. O modelo usado foi o Claude Opus 5.5. O processo teve estes passos:

1. as fórmulas, os dados inventados e a tabela de resultados esperados ficaram em arquivos no projeto;
2. as quatro regras foram pedidas ao agente de uma vez;
3. o agente escreveu o código e os testes juntos. Ele podia escrever os testes, mas não podia mudar a tabela de resultados esperados nem os dados, e os testes tinham de ler os valores esperados dessa tabela, sem valores escritos à mão;
4. o agente rodou os testes. Se algum falhasse, ele devia corrigir o código, e não os testes, e contar quantas tentativas foram necessárias;
5. no fim, o agente escreveu um registro com o que entendeu de cada regra, o que errou, o que ficou ambíguo, quantas tentativas fez e se a regra passou.

O agente também recebeu duas orientações: ser fiel ao sistema antigo, inclusive nos comportamentos estranhos, sem "consertar" as regras; e usar o tipo `Decimal` do Python para os valores em dinheiro. O `Decimal` evita pequenos erros de arredondamento dos números comuns. Os testes usam o pytest, ferramenta que roda testes automáticos e diz quais passaram (Figura 7).

Figura 7 – Saída do pytest



Quadro 5 – Tentativas, resultado e dificuldade principal por regra

Regra	Tentativas	Passou	Dificuldade principal
01	1	Sim	Um ano que só tem eventos excluídos aparece na lista de anos, mas sai pelo filtro. Esse caso não foi testado.
02	1	Sim	Não se sabe de onde vêm os campos `ValorConvertido` e `InscricaoConvertida`.
03	1	Sim	Na fórmula, foi preciso deduzir a qual lista pertence cada `ID`.
04	1	Sim	O "-" da função `Coalesce` nunca aparece.

Fonte: elaborado pela autora, com base no arquivo `estudo-caso/registro_conversao.md`.

4.7 A AVALIAÇÃO

A avaliação compara dois resultados: o que o Python calculou e o que o Power Apps mostrou. Para cada um dos 41 valores, o critério é simples: o valor está igual ao esperado ou não está. Se não está, descrevo o motivo. Além disso, registro quantos testes automáticos passaram. São 41 valores e 42 testes porque a regra 01, além dos 14 testes de contagem, tem um teste a mais, que confere se os anos aparecem do maior para o menor.

Os limites são claros: o resultado vale para quatro regras, com dados inventados, em um único sistema. Ele mostra como a conversão se comportou neste caso, e não que qualquer sistema pode ser convertido assim.

5 Resultados e Considerações Finais

Este capítulo mostra o que aconteceu no estudo de caso. Ele descreve o que foi convertido, o resultado dos testes, os desafios que observamos, o que o desenvolvedor pode fazer e os limites do estudo.

5.1 O QUE FOI CONVERTIDO

Neste estudo, quatro regras do SIS Eventos foram convertidas do Power Fx, a linguagem de fórmulas do Power Apps, para Python. As três primeiras foram planejadas desde o início, e a quarta foi feita a mais:

- regra 01: contagem dos eventos por ano e por status, sem os registros excluídos;
- regra 02: valor de cada linha de custo;
- regra 03: total previsto de custos dos eventos aprovados de um ano;
- regra 04: total realizado dos eventos aprovados de um ano.

As fórmulas foram copiadas do editor do Power Apps, uma por arquivo. A conversão foi feita pelo agente Claude Code, com o modelo Claude Opus 5.5. O código ficou em um arquivo Python, e os testes foram escritos com a ferramenta pytest.

Os dados usados são inventados. São três tabelas, com os mesmos nomes de campos das fórmulas: eventos, linhas de custo e diárias nacionais. Os casos cobrem os anos de 2025 e 2026 e 23 linhas de custo.

Para saber o resultado certo de cada caso, as fórmulas originais foram rodadas em um app vazio do Power Apps, criado só para o teste, com esses mesmos dados inventados. O app real não foi usado. A autora anotou o que o app mostrou em uma tabela de resultados esperados. Por exemplo, em 2026 o total previsto foi R$ 19.450,00 e o total realizado foi R$ 20.350,00. Os testes comparam a saída do código em Python com essa tabela.

5.2 RESULTADO DOS TESTES

Ao todo, foram 42 testes, e os 42 passaram. Nenhum teste falhou. Cada regra passou na primeira execução, ou seja, com uma tentativa só. O Quadro 6 mostra o resultado por regra.

Quadro 6 – Resultado dos testes por regra

Regra	Testes	Passaram	Tentativas
01 – eventos por ano	15	15	1
02 – valor da linha de custo	23	23	1
03 – total previsto	2	2	1
04 – total realizado	2	2	1
Total	42	42	–

Fonte: elaborado pela autora, com base no resultado do pytest.

Na regra 01, há 14 testes de contagem e um teste da ordem dos anos. Na regra 02, há um teste por linha de custo, e cada um confere o valor e o texto em reais. Nas regras 03 e 04, há um teste para 2025 e outro para 2026.

Os testes leem os valores esperados direto da tabela de resultados. Eles não têm valores esperados escritos à mão.

5.3 DESAFIOS OBSERVADOS

Mesmo com todos os testes passando, a leitura das fórmulas mostrou comportamentos que não estavam claros na descrição do sistema. Nos casos testados, o código em Python repete esses comportamentos, porque a ideia era ser fiel ao sistema antigo, e não corrigir as regras.

A regra 02 não olha o campo Excluido

A regra 02 calcula o valor da linha mesmo quando a linha está marcada como excluída. Nos casos testados, o custo 9 está excluído e mesmo assim deu R$ 800,00. Já as regras 01, 03 e 04 deixam os registros excluídos de fora. Ou seja, a regra dos registros excluídos não está em um lugar só, e cada fórmula trata isso de um jeito. Isso se liga ao desafio das regras escondidas do Quadro 1, que inclui regras espalhadas em vários lugares.

A diária nacional não é multiplicada pelas pessoas

Na regra 02, a diária nacional da lista é calculada como quantidade de diárias × valor da diária, sem multiplicar pela quantidade de pessoas. Nos casos testados, o custo 4 tem 2 pessoas e deu R$ 800,00, que é 2 diárias × 400. Esse comportamento só aparece quando se lê a fórmula. Por isso, ligamos esse ponto ao desafio das regras escondidas do Quadro 1.

A diária excluída ainda é usada

A regra 02 busca o valor da diária na lista de diárias sem tirar as que estão excluídas. Nos casos testados, o custo 5 usa uma diária excluída e deu R$ 1.000,00. A regra 03, por outro lado, ignora as diárias excluídas. É o mesmo caso da regra dos excluídos espalhada em várias fórmulas, e por isso também ligamos esse ponto às regras escondidas do Quadro 1.

O “-” da regra 04 nunca aparece

A regra 04 usa a função `Coalesce` para mostrar “-” quando não há valor. Mas o primeiro argumento sempre começa com “R$”, então nunca fica vazio, e o “-” não aparece. O código em Python também não devolve “-”. Não encontramos ligação direta desse ponto com os desafios do Quadro 1.

De onde vêm os valores convertidos

Na regra 02, a diária internacional e a inscrição não fazem conta. Elas só mostram dois campos já guardados na linha: `ValorConvertido` e `InscricaoConvertida`. Neste estudo, não sabemos de onde vêm esses valores. Eles podem ser calculados em outra parte do sistema, que não foi vista. Isso parece com o desafio das regras escondidas do Quadro 1, no qual o agente não percebeu partes vizinhas do sistema. Mas, como não sabemos a origem dos campos, essa ligação não pôde ser confirmada.

A mesma conta feita de dois jeitos

O valor das linhas de custo aparece em duas fórmulas: na regra 02, linha por linha, e na regra 03, dentro do total. As duas fazem a conta de jeitos diferentes. A regra 03 multiplica a diária nacional pelas pessoas e ignora a diária excluída, e a regra 02 não faz nenhuma das duas coisas.

Por isso, em 2026, a parte do evento 1 no total da regra 03 é 6.350, enquanto a soma das linhas desse evento na regra 02, sem o custo 9, é 6.550. Isso se liga ao desafio das regras escondidas do Quadro 1, porque a mesma regra está em mais de um lugar.

O texto em reais

O app mostra os valores como texto, no formato "R$ 1.234,56", que vem da função `Text()` do Power Fx. O código em Python precisou montar esse mesmo texto. Também ficou uma dúvida: quando a regra 02 não tem resultado, o app parece mostrar só "R$", porque o `Text()` de um valor vazio dá texto vazio. Esse caso não foi testado. Não encontramos ligação direta desse ponto com os desafios do Quadro 1.

5.4 O QUE O DESENVOLVEDOR PODE FAZER

A partir do que observamos neste estudo, sugerimos alguns cuidados. Eles vêm de um caso só e de poucas regras, então são sugestões, e não regras para qualquer sistema.

- **Listar onde cada regra aparece.** Neste estudo, a regra dos registros excluídos estava em várias fórmulas, e cada uma tratava isso de um jeito. Antes de converter, vale procurar todas as fórmulas que usam o campo `Excluido` e anotar quais olham esse campo e quais não olham.
- **Pedir ao agente que aponte regras duplicadas.** O valor das linhas de custo aparecia nas regras 02 e 03, com contas diferentes. Pedir ao agente que compare as fórmulas e mostre onde a mesma conta é feita de mais de um jeito pode ajudar a achar essas diferenças antes dos testes.
- **Perguntar à equipe a origem dos campos guardados.** Neste estudo, não soubemos de onde vêm `ValorConvertido` e `InscricaoConvertida`. Perguntar ao responsável pelo sistema antes de converter evita copiar um campo sem saber como ele é calculado.
- **Escrever testes com centavos e casos de borda.** Nos casos testados, todos os valores eram inteiros, e vários casos ficaram sem teste. Incluir valores com centavos, anos sem eventos aprovados e linhas sem nenhum caso que se aplique ajudaria a testar o arredondamento e os comportamentos que não apareceram.
- **Conferir o texto na tela.** O texto em reais foi montado a partir do formato das fórmulas, e não copiado da tela. Copiar alguns textos direto do app ajudaria a confirmar o formato, o caso do "R$" sozinho e o "-" da regra 04.

5.5 LIMITAÇÕES DO ESTUDO

Este estudo tem limites que precisam ser levados em conta.

Primeiro, os dados inventados e o roteiro do app de teste foram criados por nós, com a ajuda do mesmo agente que depois fez a conversão. Esse agente também fez antes as contas à mão dos totais. Ou seja, o agente teve acesso aos resultados esperados.

Segundo, a coluna com o texto mostrado na tela, como “R$ 19.450,00”, não foi copiada da tela. Ela foi montada a partir do formato das fórmulas, com os valores anotados pela autora.

Terceiro, todos os valores dos dados são inteiros. Por isso, nenhum caso testa centavos nem o arredondamento. Também ficaram sem teste:

- um ano que só tem eventos excluídos;
- um ano sem eventos aprovados;
- uma linha de custo com campos de mais de um tipo preenchidos;
- uma linha da regra 02 sem nenhum caso que se aplique, ou com o valor convertido vazio.

Quarto, foram convertidas só quatro regras de um sistema maior.

Quinto, o tempo gasto em cada etapa não foi registrado. Por isso, o custo em tempo do uso do agente não foi avaliado neste estudo.

Por tudo isso, o fato de todas as regras terem passado na primeira tentativa não permite concluir que a conversão é simples. Nos casos testados, o código em Python deu os mesmos resultados do app de teste. Mas os comportamentos descritos na Seção 5.3 só foram vistos porque as fórmulas foram lidas com atenção, e não porque algum teste falhou.

5.5.1 Pontos não verificados

Alguns pontos das regras ficaram sem confirmação neste estudo:

- não verificamos com o responsável pelo sistema se a diária nacional deve ser calculada por pessoa. Na regra 02, a conta da linha não multiplica pela quantidade de pessoas, mas na regra 03 o total multiplica;

- não verificamos de onde vêm os campos de valor convertido, `ValorConvertido` e `InscricaoConvertida`;
- não verificamos a origem da variável do ano (`VarAnoExercicioSelecionado`), usada nas regras 03 e 04. No app de teste, ela foi definida à mão.

6 Conclusão

Este trabalho teve como objetivo entender os desafios de atualizar sistemas legados com agentes de IA, do ponto de vista do desenvolvimento.

Na literatura, encontramos desafios em seis pontos: tamanho do sistema, complexidade das funcionalidades, regras escondidas, atalhos do agente, linguagens antigas e custo. Nos estudos lidos, o resultado do agente foi verificado de alguma forma, e os testes foram uma das formas usadas para isso.

No estudo de caso, quatro regras do SIS Eventos foram convertidas do Power Fx para Python, com o agente Claude Code. Os resultados esperados vieram de um app vazio do Power Apps, com dados inventados. Foram 42 testes, e os 42 passaram. Cada regra passou na primeira tentativa.

Mesmo assim, a leitura das fórmulas mostrou comportamentos escondidos. A regra 02 não olha o campo de registro excluído, não multiplica a diária nacional pelas pessoas e usa uma diária excluída. A mesma conta aparece nas regras 02 e 03, feita de jeitos diferentes. E não sabemos de onde vêm dois campos de valor convertido. Esses pontos não apareceram porque um teste falhou. Eles apareceram porque as fórmulas foram lidas com atenção.

Respondendo à pergunta de pesquisa: neste estudo, o principal desafio observado foi o das regras escondidas do Quadro 1. Nos casos testados, o agente deu os mesmos resultados do sistema atual. A dificuldade estava em saber se os testes cobriam o suficiente e em entender regras que estavam espalhadas ou que dependiam de partes do sistema que não foram vistas. Os outros desafios do quadro não foram observados ou não foram medidos neste estudo.

O estudo tem limites. Os dados foram inventados, e o agente que fez a conversão também ajudou a criar os dados e viu os resultados esperados. Foram só quatro regras de um sistema maior, e nenhum caso testou centavos ou arredondamento. Por isso, passar de primeira não permite dizer que a conversão é fácil ou difícil em geral. Alguns pontos das regras também ficaram sem confirmação, e eles estão listados na Seção 5.5.1.

Como trabalhos futuros, sugerimos converter mais regras do mesmo sistema, usar dados reais com a autorização do responsável, repetir o estudo com outros agentes e medir o tempo gasto em cada etapa.

Referências

ALVES, I. d. S. R.; POLITOWSKI, C.; MONTANDON, J. E. Legacy System Modernization with Coding Agents: A Case Study. 2026. ArXiv:2608.28972. Disponível em: <https://arxiv.org/abs/2608.28972>.

CHAKARAVARTHY, V. et al. Enterprise-scale COBOL-to-Java translation: LLMs augmented with program analysis. In: Proceedings of the 48th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '26). Rio de Janeiro, Brasil: [s.n.], 2026.

DIGGS, C. et al. Leveraging LLMs for Legacy Code Modernization: Challenges and Opportunities for LLM-Generated Documentation. 2024. ArXiv:2411.14971. Disponível em: <https://arxiv.org/abs/2411.14971>.

LI, X. et al. RepoMod-Bench: A Benchmark for Code Repository Modernization via Implementation-Agnostic Testing. 2026. ArXiv:2602.22518. Disponível em: <https://arxiv.org/abs/2602.22518>.

LIU, L. et al. MigrationBench: Repository-level code migration benchmark from java 8. In: Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '26). Jeju, Coreia do Sul: [s.n.], 2026. ArXiv:2505.09569.

MAY, V. et al. FreshBrew: A Benchmark for Evaluating AI Agents on Java Code Migration. 2025. ArXiv:2510.04852. Disponível em: <https://arxiv.org/abs/2510.04852>.

PAN, R. et al. Lost in translation: A study of bugs introduced by large language models while translating code. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE '24). Lisboa, Portugal: [s.n.], 2024.